EXPERIMENT 3

Academic Year: 2026-27 Year and Sem: TE V
Subject: DEV Lab Course Code:

PROBLEM STATEMENT:

To configure and implement a Continuous Integration (CI) pipeline using GitHub Actions.

Requirements:

Git installed , GitHub account, Java JDK 17, Maven

THEORY:

Continuous Integration (CI) is a software development practice in which developers regularly merge their code changes into a shared repository. Whenever new code is pushed to the repository, an automated process is triggered to build, test, and verify the application. This helps identify errors early in the development cycle and ensures that the software remains stable.

Instead of manually compiling and testing the project after every change, CI automates these tasks, saving time and reducing human errors.

What is GitHub Actions?

GitHub Actions is a cloud-based automation service provided by GitHub that allows developers to automate software development workflows directly from a GitHub repository. It can automatically perform tasks such as building applications, running tests, deploying software, and sending notifications whenever specific events occur, such as a code push or pull request.

GitHub Actions eliminates the need for a separate CI server because the workflow runs automatically on GitHub's hosted virtual machines.

What is a Workflow?

A **workflow** is a sequence of automated steps that GitHub Actions performs. It is defined in a YAML file stored inside the project under the .github/workflows directory.

A workflow specifies:

- When the automation should start (trigger).
- What environment should be used.
- What tasks should be executed.

For example, a workflow may automatically compile a Java project and execute tests whenever code is pushed to the repository.

node -v

npm -v

Git --version

Step 1: Create a Node.js Project

Open Terminal.

Create a new folder. :mkdir github-actions-demo

Go inside the folder.: cd github-actions-demo

Initialize Node project.

npm init -y

npm init

npm install

Step 2: Create Project Files

Create three files.

app.js

app.test.js

package.json

Project structure:

github-actions-demo

│

├── app.js

├── app.test.js

├── package.json

Inside the project folder, type:

touch app.js app.test.js

Step 3: Install Jest

Install Jest.

npm install --save-dev jest

Step 4: Edit package.json

n this step, you need to modify the package.json file so that the npm test command runs **Jest**. The notes indicate that the workflow later executes npm test, so the test script must be configured accordingly.

Click on **package.json**.

Initially, it looks something like this:

```
{
  "name": "github-actions-demo",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  }
}
```

3. Find the scripts section

Locate this:

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
}
```

4. Replace it with

```
"scripts": {
  "test": "jest"
}
```

5. Your package.json should now look similar to this

```
{
  "name": "github-actions-demo",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "test": "jest"
  },
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "jest": "^30.0.0"
  }
}
```

Note: The Jest version number (^30.0.0 or similar) may differ depending on the version installed on your system.

Step 5: Write app.js

nano app.js

```
function add(a, b) {
    return a + b;
}


module.exports = add;
```

Save the file.

Step 6: Write app.test.js

```
const add = require('./app');


test('adds 2 + 3 to equal 5', () => {
    expect(add(2,3)).toBe(5);
});
```

Save the file.

Step 7: Test Locally

Run

```
npm test
```

Expected output

```
PASS ./app.test.js


Test Suites: 1 passed
Tests: 1 passed
```

Step 8: Initialize Git Repository

```
git init
```

Step 9: Add Files

```
git add .
```

Commit the project.

```
git commit -m "Initial Commit"
```

Step 10: Create GitHub Repository

Login to GitHub.

Click **New Repository**.

Repository Name

```
github-actions-demo
```

Click **Create Repository**.

Step 11: Connect Local Repository

Copy the GitHub repository URL.

Example

git remote add origin https://github.com/username/github-actions-demo.git

Push the code.

git branch -M main

git push -u origin main

In this step, you need to create the .github/workflows folder and the build.yml file. GitHub automatically checks this folder for workflow files.

Step 1: Go to your project folder

cd github-actions-demo

Verify your current location:

pwd

Step 2: Create the .github folder

mkdir .github

Step 3: Go inside .github

cd .github

Step 4: Create the workflows folder

mkdir workflows

step 5: Go inside the workflows folder

cd workflows

Step 6: Create the build.yml file

touch build.yml

Step 7: Verify the file exists

ls

Output:

build.yml

Step 8: Open build.yml using nano

nano build.yml

Paste the following workflow:

```
name: Node.js CI Pipeline


on:
  push:
    branches:
      - main


jobs:
  build:
    runs-on: ubuntu-latest


    steps:
      - name: Checkout Source Code
        uses: actions/checkout@v4


      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '24'
```

```
    - name: Install Dependencies
      run: npm install

    - name: Run Tests
      run: npm test
```

run: npm install

name: Run Tests using run: npm test

Step 9: Save the file

Press:

Ctrl + O

Press **Enter**.

Then exit Nano:

Ctrl + X

Step 10: Return to the project folder

cd ../..

Now verify the folder structure:

find .

Expected output:

```
.
├── .github
│   └── workflows
│       └── build.yml
├── app.js
├── app.test.js
├── package.json
└── node_modules
```

Step 11: Add Workflow File

git add .

Commit

git commit -m "Added GitHub Actions Workflow"

run : gh auth login

Push

git push origin main

Step 12: Open GitHub

Open your repository.

Click

Actions

GitHub automatically starts the workflow after the push.

Step 13: Check Workflow Status

You will see

Node.js CI Pipeline

Click it.

Click

build

You will see logs for each step:

Checkout Source Code

Setup Node.js

Install Dependencies

Run Tests

A green tick indicates the workflow completed successfully.

Conclusion :

By performing this experiment, students understood the fundamentals of Agile Software Development and the Scrum framework using Jira. They learned how to create a Scrum project, define epics and user stories, organize a product backlog, plan and start a sprint, assign tasks, and monitor project progress using the Scrum board and reports. This experiment provided practical experience in Agile project management and demonstrated how Jira supports collaboration, task tracking, and efficient software development in real-world environments